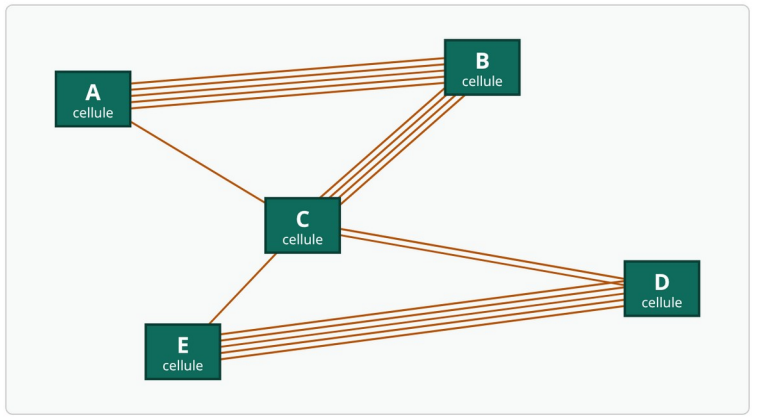
Bipartition initiale

Avant d'améliorer une coupe, il faut une première coupe, appelée la bipartition initiale

D'une puce à un graphe

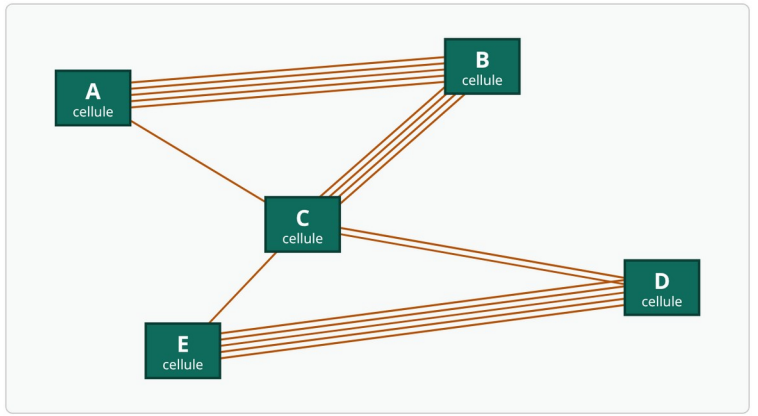
- Une puce est faite de petites briques appelées cellules, reliées par des fils
- Notre exemple a cinq cellules, nommées de A à E
- Certaines paires partagent beaucoup de fils, d'autres un seul



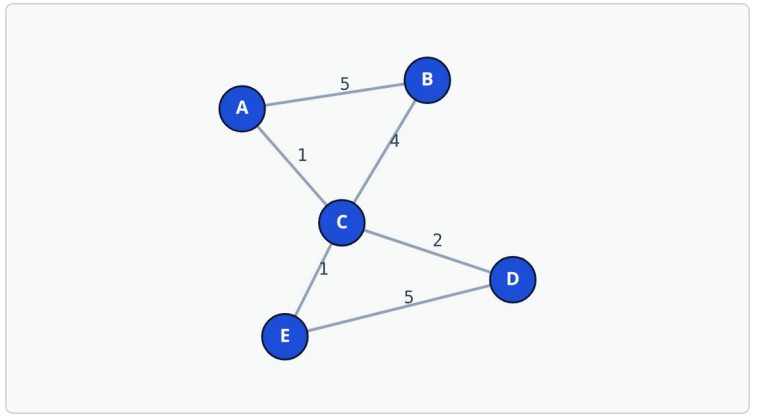
Les cellules deviennent des points, les fils des traits

- On dessine chaque cellule comme un point, appelé nœud
- Un trait entre deux nœuds est une arête
- Son nombre, le poids, compte les fils entre les deux cellules
- A–B et D–E sont les plus lourdes, avec cinq fils chacune

Sur la puce : des cellules reliées par des fils



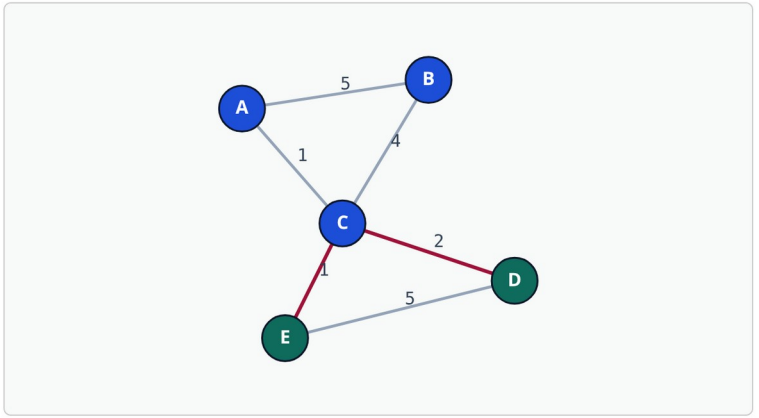
En graphe : poids = nombre de fils



Deux nombres : coupe et équilibre

- Couper les cellules en deux côtés s'appelle une bipartition
- Les arêtes qui passent d'un côté à l'autre sont coupées, et dessinées en rouge
- La taille de coupe additionne leurs poids : ici, deux plus un font trois

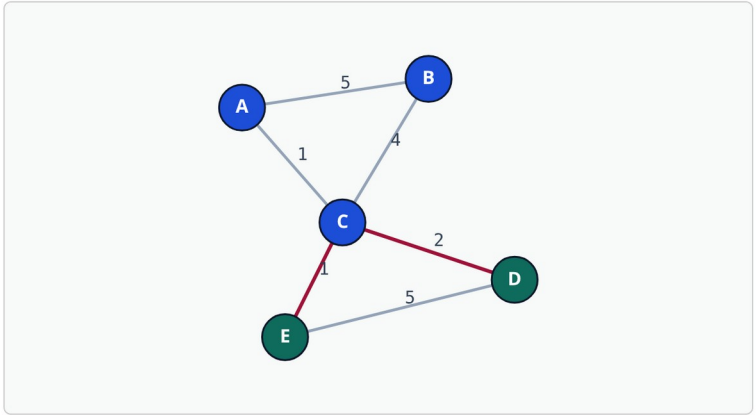
A B C | D E : coupe C–D 2 + C–E 1 = 3. Tailles 3 et 2 : valide.



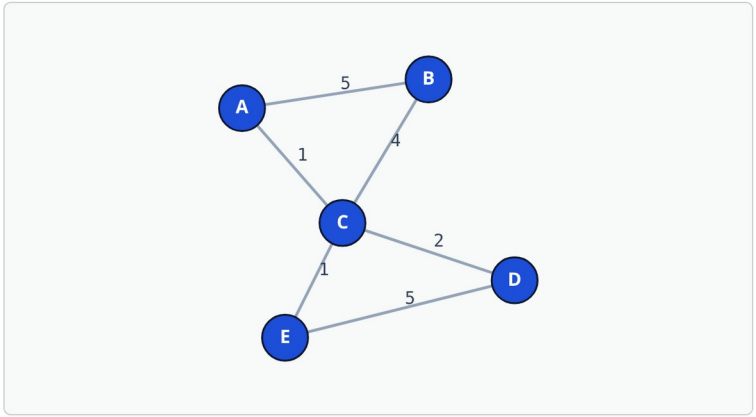
L'équilibre : aucun côté trop gros

- Tout mettre du même côté donne une coupe de zéro, mais cela ne sert à rien
- On vérifie donc aussi l'équilibre : combien de cellules chaque côté contient
- Une partition est valide quand aucun côté ne dépasse trois cellules sur cinq

A B C | D E : coupe C–D 2 + C–E 1 = 3. Tailles 3 et 2 : valide.



Les cinq du même côté : coupe 0. Tailles 5 et 0 : non valide.



Pourquoi une partition de départ ?

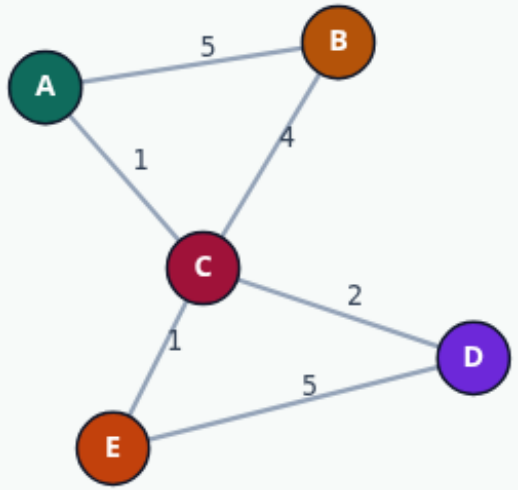
- Les labs quatre et cinq enseignent deux méthodes célèbres, appelées KL et FM
- Elles améliorent une partition en déplaçant des cellules d'un côté à l'autre
- Mais KL et FM ont besoin d'une partition valide pour démarrer
- Pensez à former deux équipes : d'abord deux équipes au hasard, puis des échanges
- Un bon premier découpage laisse moins d'échanges à faire
- Faites une pause et devinez : le hasard peut-il aussi trouver une coupe de trois ?

Trois façons de construire une première partition

BIPARTITION INITIALE

Trois façons de construire une première partition

Étape 1 / 5 · empty · module01-03-initial-bipartition



Explication

Trois recettes courtes construisent une première partition : aléatoire, gloutonne et par croissance. Chaque nœud va d'un des deux côtés, et aucun côté ne peut contenir plus de 3 des 5 nœuds.

- Recettes : aléatoire, glouton, croissance
- Chaque nœud sur exactement un côté
- Ici : des côtés de 2 et 3 nœuds

```
5 nœuds, 6 arêtes, poids total 18
tailles valides : 2 | 3
```

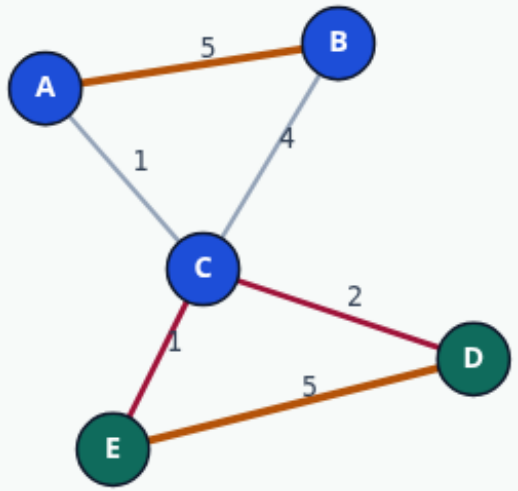
Trois façons de construire une première partition

Aléatoire, graine 5 : ABC|DE, coupe 3

BIPARTITION INITIALE

Aléatoire, graine 5 : ABC|DE, coupe 3

Étape 2 / 5 · random-lucky · module01-03-initial-bipartition



Explication

L'aléatoire mélange les nœuds comme des cartes et en distribue deux à un côté. La graine fixe le mélange. La graine 5 donne D et E : seules C–D et C–E sont coupées, taille de coupe 3.

- Mélanger les noms, en donner 2 à un côté
- Même graine, même partition à chaque fois
- Graine 5 → ABC|DE, coupe 3

```
méthode : aléatoire graine=5
partition : ABC|DE
taille de coupe : 3 = C–D 2 + C–E 1
```

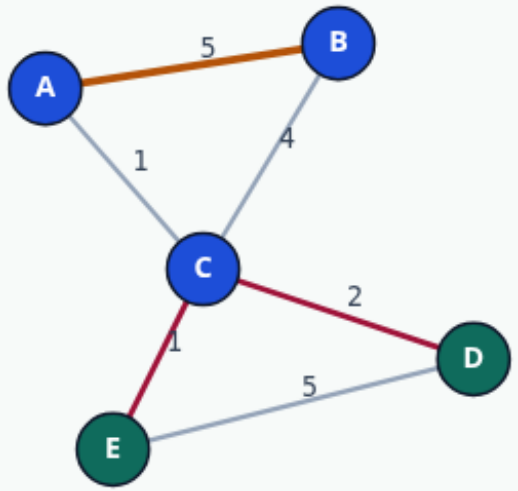
Aléatoire, graine 5 : ABC|DE, coupe 3

Glouton : ABC|DE, coupe 3

BIPARTITION INITIALE

Glouton : ABC|DE, coupe 3

Étape 3 / 5 · greedy · module01-03-initial-bipartition



Explication

Le glouton garde l'arête la plus lourde A–B d'un seul côté (égalité avec D–E : le plus petit nom gagne). D, le moins lié à A et B, démarre l'autre côté. C rejoint A et B ; E rejoint D : cela coupe 1 au lieu de 5.

- Ancre : A–B, poids 5
- L'autre côté démarre avec D
- Chaque nœud va là où il ajoute le moins de coupe

```
méthode : glouton
partition : ABC|DE
taille de coupe : 3
```

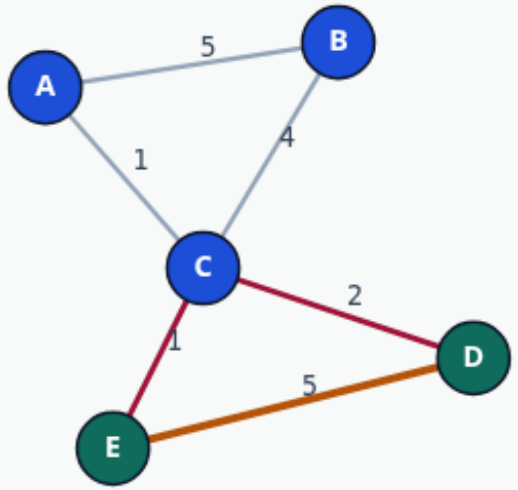
Glouton : ABC|DE, coupe 3

Croissance depuis D : ABC|DE, coupe 3

BIPARTITION INITIALE

Croissance depuis D : ABC|DE, coupe 3

Étape 4 / 5 · grow-d · module01-03-initial-bipartition



Explication

La croissance démarre un côté avec D et ajoute le nœud au bout de l'arête la plus lourde qui en sort : E, poids 5. Le côté contient alors 2 des 5 nœuds, donc il s'arrête. A, B et C forment l'autre côté : coupe 3.

- Nœud de départ : D
- Arête la plus lourde qui sort : D–E (5)
- 2 nœuds sur 5 : stop

```
méthode : croissance depuis D
partition : ABC|DE
taille de coupe : 3
```

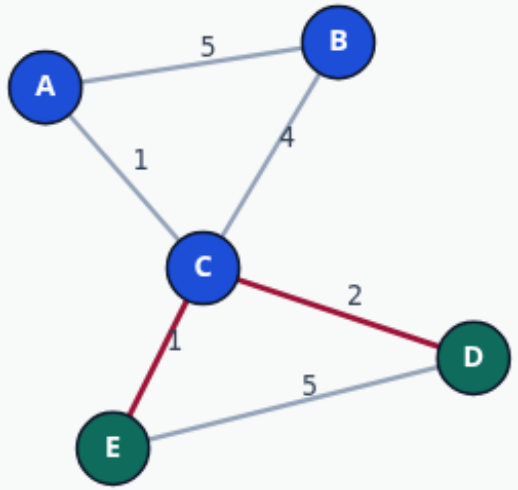
Croissance depuis D : ABC|DE, coupe 3

Les trois trouvent une coupe de 3. Coup de chance ?

BIPARTITION INITIALE

Les trois trouvent une coupe de 3. Coup de chance ?

Étape 5 / 5 · takeaway · module01-03-initial-bipartition



Explication

L'aléatoire avec la graine 5, le glouton et la croissance depuis D donnent la même partition, ABC|DE, avec une coupe de 3. Est-ce une bonne méthode, ou de la chance ?

- Aléatoire, graine 5 → coupe 3
- Glouton → coupe 3
- Croissance depuis D → coupe 3

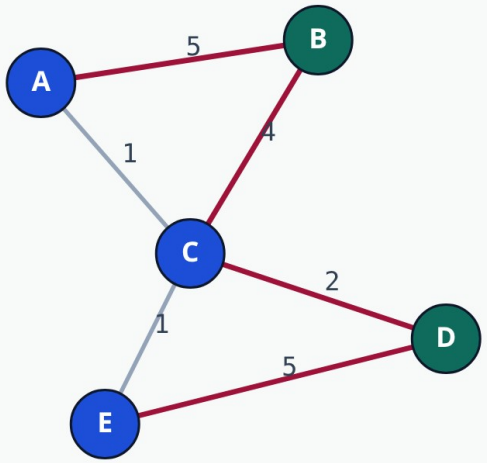
```
même partition : ABC|DE, coupe 3
chance, ou bonne méthode ?
```

Les trois trouvent une coupe de 3. Coup de chance ?

Exemple détaillé : une graine malchanceuse

- Un seul mélange malchanceux peut rendre la coupe plus de cinq fois pire
- Donc oui, c'était en partie de la chance
- Avec la graine trois, B et D tombent d'un côté, et A, C et E de l'autre

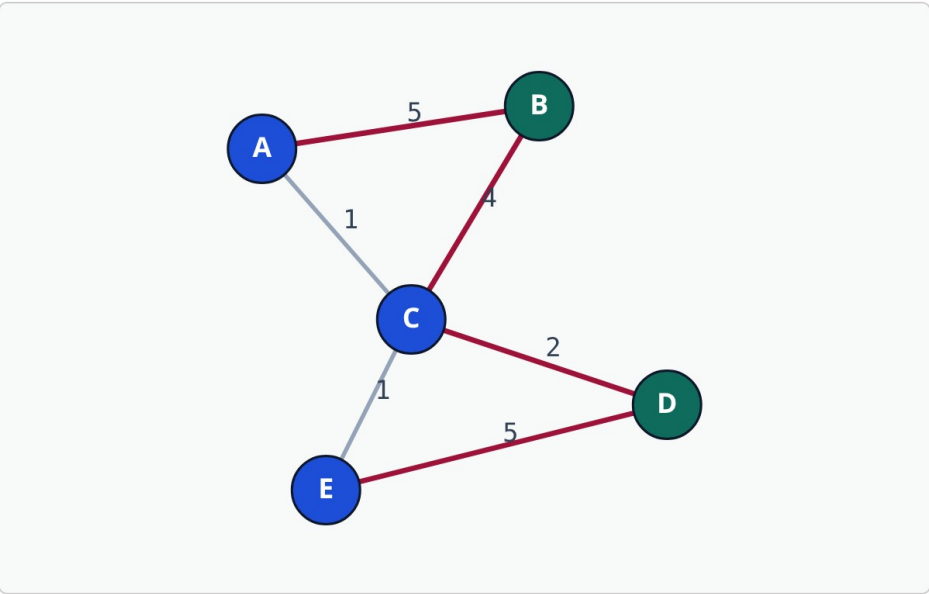
Aléatoire, graine 3 : A C E | B D



Additionner la coupe

- Additionnons la coupe, arête par arête
- A–B cinq, B–C quatre, C–D deux, et D–E cinq
- Cela fait seize

Aléatoire, graine 3 : A C E | B D

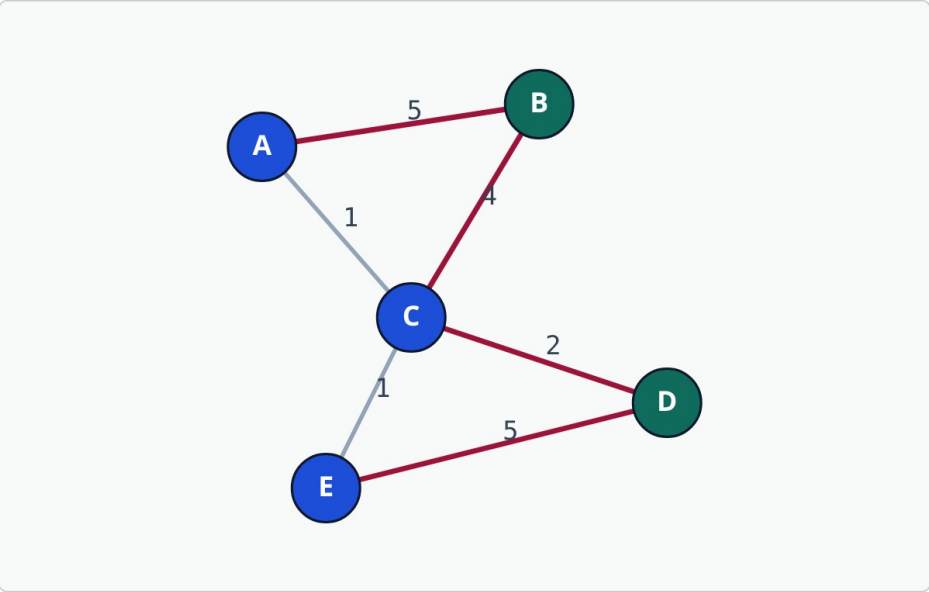



```
1. Coupées (rouge) : A–B, B–C, C–D, D–E
2. Coupe : 5 + 4 + 2 + 5 = 16
```

La pire partition valide

- Les deux arêtes lourdes sont coupées : c'est la pire partition valide qui existe
- Sur seize graines, la coupe va de trois à seize
- Seules trois de ces graines trouvent la meilleure coupe, trois

Aléatoire, graine 3 : A C E | B D

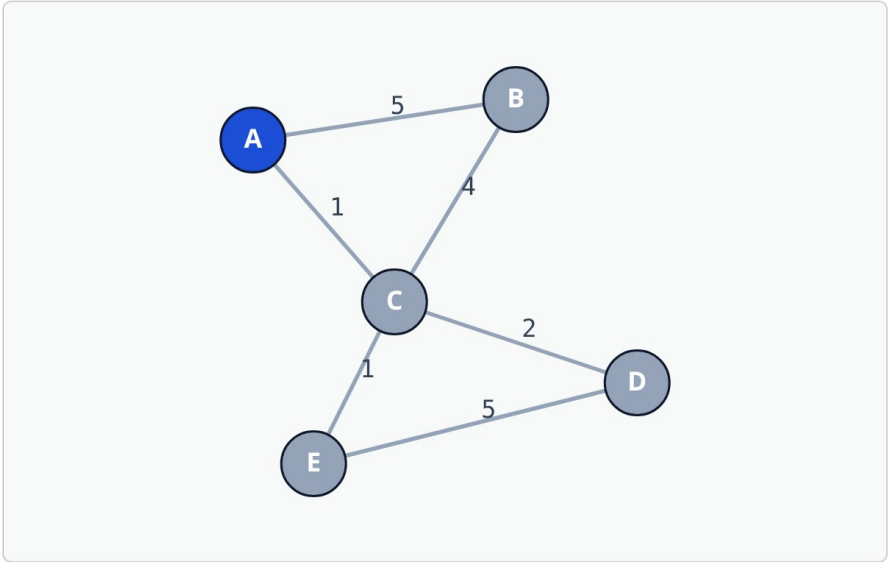


1. Coupées (rouge) : A–B, B–C, C–D, D–E
2. Coupe : 5 + 4 + 2 + 5 = 16
3. **Tailles 3 et 2 : valide, mais le pire départ**

La croissance dépend du nœud de départ

- La croissance n'a pas de hasard, mais elle a un choix : le nœud de départ
- Partons de A au lieu de D

Croissance depuis A

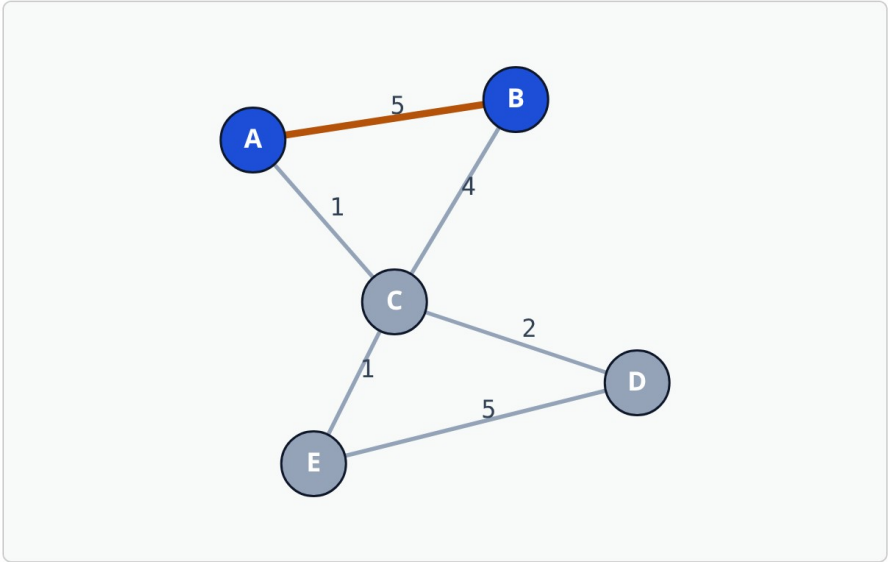



```
1. Le côté démarre avec A
```

Croissance depuis A, étape par étape

- L'arête la plus lourde qui sort de A va vers B, avec un poids de cinq, donc B rejoint A
- Avec deux cellules, le côté est plein, et il s'arrête

Croissance depuis A

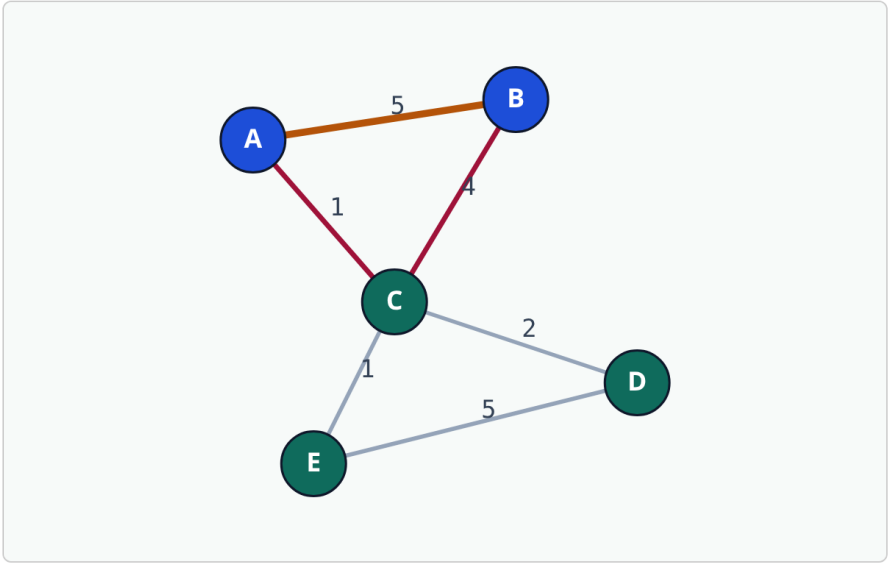



```
1. Le côté démarre avec A
2. Arête la plus lourde depuis A : A–B 5, on
ajoute B
3. 2 nœuds sur 5 : le côté est plein, stop
```

Croissance depuis A : AB|CDE, coupe 5

- C, D et E forment l'autre côté, donc A–C et B–C sont coupées
- Un plus quatre font cinq, ce qui est pire que trois
- Partir de C est encore pire : une coupe de neuf

Croissance depuis A




```
1. Le côté démarre avec A
2. Arête la plus lourde depuis A : A–B 5, on
ajoute B
3. 2 nœuds sur 5 : le côté est plein, stop
4. Coupe : A–C 1 + B–C 4 = 5
```

Garder le meilleur départ valide

- La solution est simple : essayer plusieurs départs, garder le meilleur
- Construisez quelques graines aléatoires, le glouton, et la croissance depuis chaque nœud
- Jetez toute partition qui enfreint la règle de taille, puis gardez la plus petite coupe
- Ici, le glouton et les cinq croissances donnent trois, cinq, cinq, neuf, trois et trois
- Le meilleur est donc trois : A, B et C contre D et E
- Faites une pause et devinez : une coupe plus petite peut-elle être le mauvais choix ?

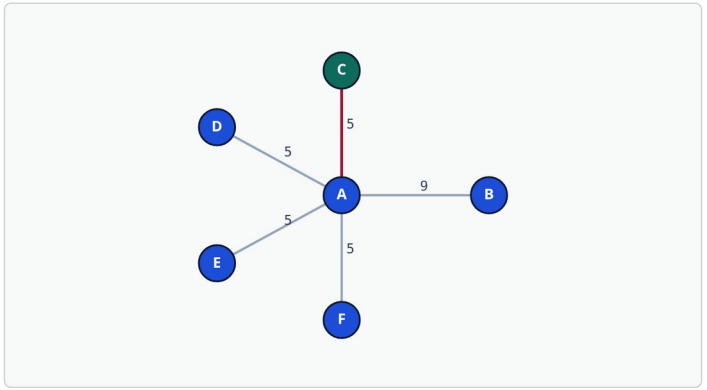
Garder le meilleur départ valide — à essayer

```
best_start(graph):
  starts = random(seed) pour quelques graines
         + greedy
         + grow(x) pour chaque nœud x
  jeter chaque départ qui n'est pas valide
  renvoyer le départ de plus petite taille de coupe
```

Piège 1 : pas de limite de taille

- Trois pièges peuvent tromper une méthode qui a l'air raisonnable
- Et oui, une coupe plus petite peut être le mauvais choix
- Voici une étoile : A au milieu, relié à cinq autres cellules
- Un ancien glouton sans limite de taille ajoutait toujours des cellules près de A
- Il finissait avec cinq cellules contre une, et une coupe de cinq

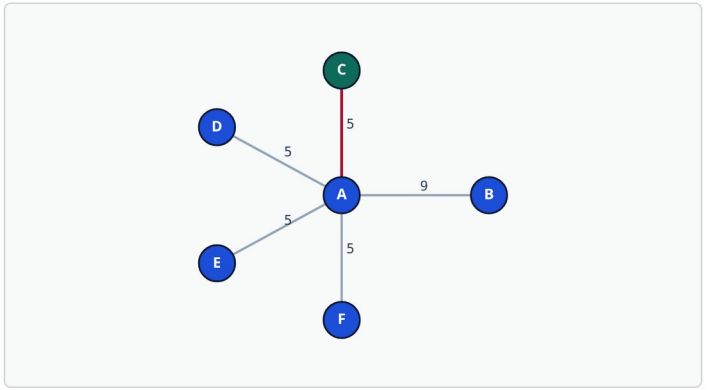
Ancien glouton, sans limite : A B D E F | C. Tailles 5 et 1, coupe 5. Non valide.



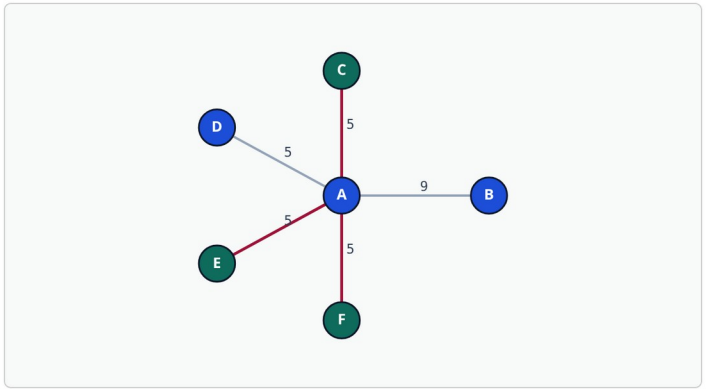
Piège 1 : la limite de taille règle le problème

- Avec six cellules, il en faut trois par côté, donc cette partition n'est pas valide
- Notre glouton arrête chaque côté à trois cellules, et obtient quinze
- Aucune partition valide de cette étoile ne fait mieux

Ancien glouton, sans limite : A B D E F | C. Tailles 5 et 1, coupe 5. Non valide.



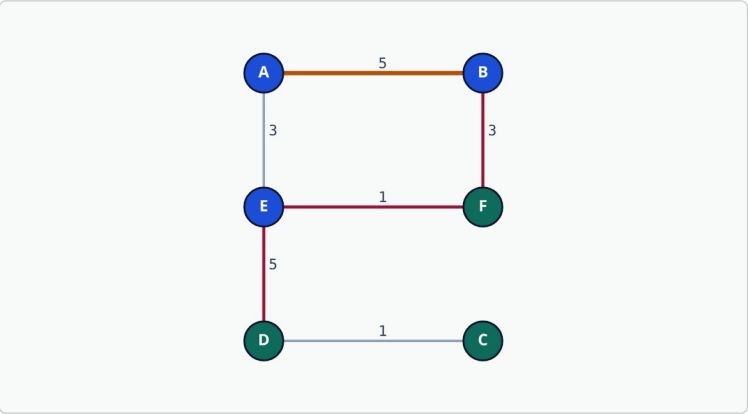
Glouton avec la limite : A B D | C E F. Tailles 3 et 3, coupe 15. Valide.



Piège 2 : une égalité peut décider du résultat

- Dans ce graphe, A–B et D–E pèsent cinq toutes les deux
- Le glouton doit en choisir une pour démarrer
- La règle prend le plus petit nom, A–B, et cela donne une coupe de neuf

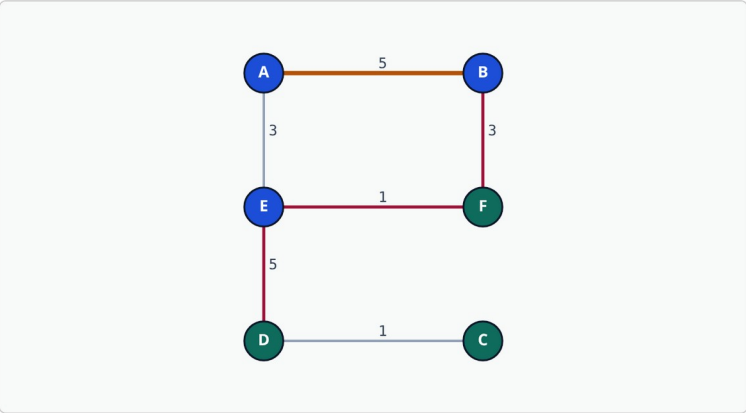
Ancre A–B (la règle) : A B E | C D F, coupe 9



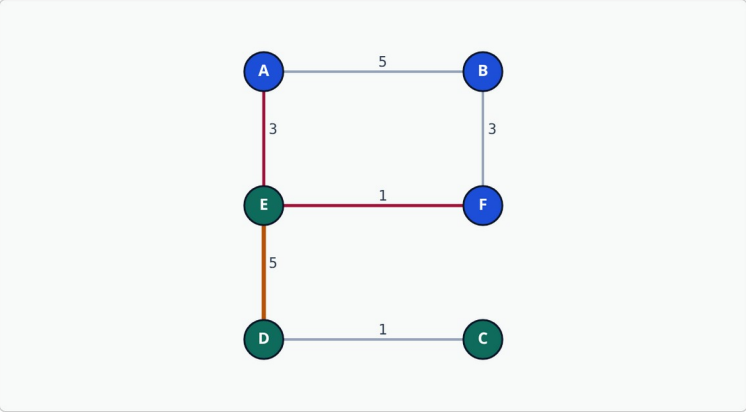
Piège 2 : l'autre ancre

- Démarrer de D–E donne une coupe de quatre
- Une simple règle de départage a donc plus que doublé la coupe
- La croissance depuis C trouve aussi quatre

Ancre A–B (la règle) : A B E | C D F, coupe 9



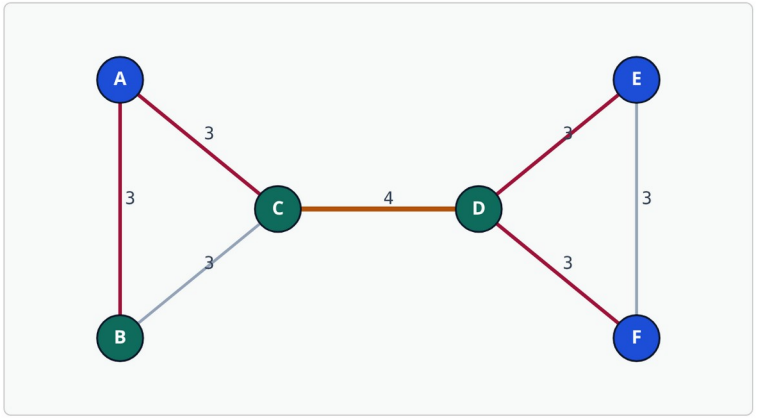
Ancre D–E : A B F | C D E, coupe 4



Piège 3 : le fil le plus lourd n'est pas toujours à l'intérieur

- Voici deux triangles, reliés par un seul pont, C–D, de poids quatre
- Chaque arête de triangle pèse trois
- Le glouton démarre de l'arête la plus lourde, qui est ici le pont lui-même
- C et D commencent donc du même côté, et la coupe finit à douze

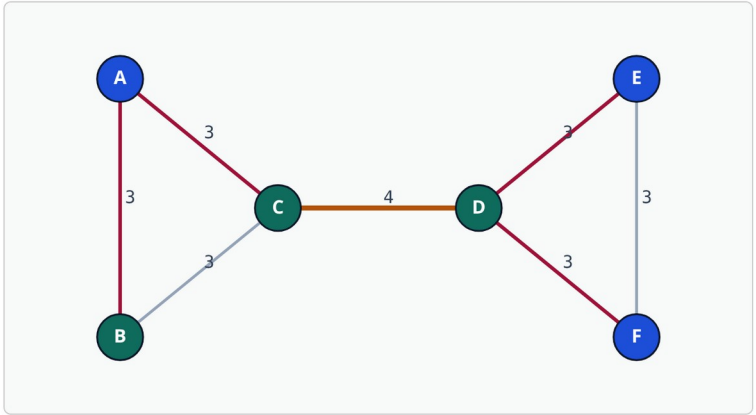
Le glouton s'ancre sur le pont C–D : A E F | B C D, coupe 12



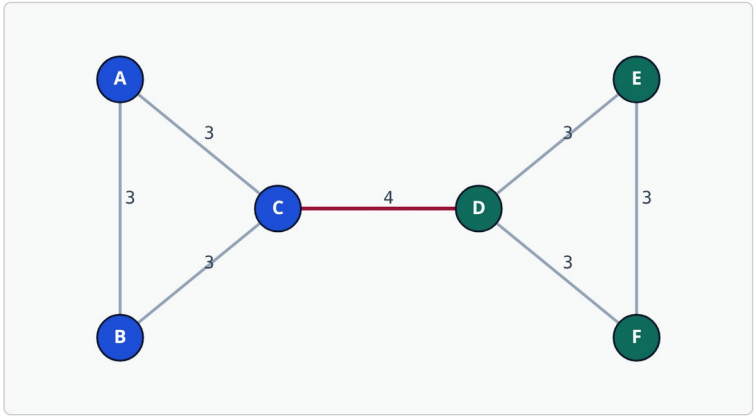
Piège 3 : la croissance garde les triangles

- La croissance depuis A garde son triangle ensemble, et ne coupe que le pont : quatre
- Faites une pause et devinez : la croissance est-elle toujours meilleure que le glouton ?

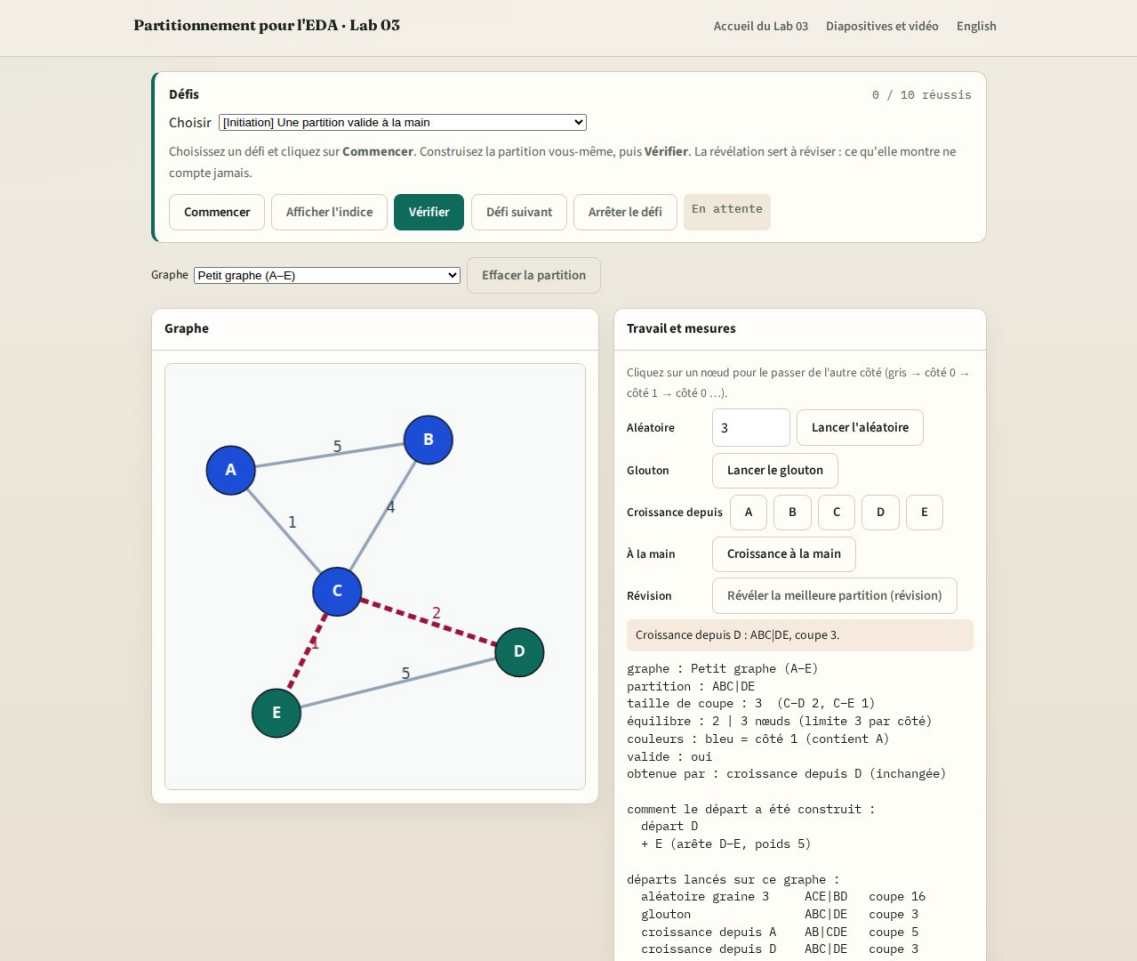
Le glouton s'ancre sur le pont C–D : A E F | B C D, coupe 12



Croissance depuis A : A B C | D E F, coupe 4



Parcours lab dans le navigateur



Lab dans le navigateur, état de départ

Parcours implémentation

```
wsl — module01-03-initial-bipartition/examples/.

# Track A — real Unix
# real shell session (Track A)

$ python3 initial_bipartition_check.py | grep PASS | head -14
PASS  tiny graph, random: seeds 0-15 give the golden splits and cuts
PASS  tiny graph, random: node order does not matter (input reversed)
PASS  tiny graph, random: every seed gives a legal 2|3 split
PASS  tiny graph, greedy: ABC|DE cut 3
PASS  star graph, greedy: ABD|CEF cut 15
PASS  tie graph, greedy: ABE|CDF cut 9
PASS  bridge graph, greedy: AEF|BCD cut 12
PASS  tiny graph, greedy: input reversed gives the same split
PASS  tie graph, greedy: input reversed gives the same split
PASS  star graph, greedy: both sides stay within ceil(n/2) nodes (3|3)
PASS  tiny graph, grow: from each node gives the golden split and cut
PASS  bridge graph, grow: from each node gives the golden split and cut
PASS  tiny graph, grow: input reversed gives the same splits
PASS  tiny graph, legality: everything on one side (cut 0) is NOT legal

$ python3 initial_bipartition_check.py | tail -1
all goldens match
```

Une vraie session de terminal

Parcours implémentation — à essayer

```
# afficher toutes les bonnes réponses
python3 examples/initial_bipartition_check.py --show

# vérifier vos propres fonctions
python3 examples/initial_bipartition_check.py --impl my_init.py
```

À retenir

- L'aléatoire est rapide, mais il dépend de la chance
- Le glouton garde les arêtes lourdes dedans, mais égalités et ponts le trompent
- La croissance dépend de son nœud de départ
- Donnez l'équilibre avec chaque coupe, et ne laissez aucun côté dépasser sa limite
- Départagez par le nom, pas par l'ordre du fichier d'entrée
- Donc : essayer plusieurs départs, garder le meilleur départ valide

À vous de jouer

- Choisissez un parcours : le lab dans le navigateur, Python, ou les deux
- Cochez chaque case de la liste de contrôle, puis faites le court quiz
- La suite, c'est Kernighan–Lin, qui part de votre partition et l'améliore
- Faites une pause et devinez : de combien peut-il réduire la coupe malchanceuse de seize ?
- Le lab quatre vous le montrera